

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3092 -Deep Learning y Sistemas Inteligentes

SECCIÓN 30

Ing. JAVIER JOSUÉ FONG GUZMÁN



Laboratorio 5

Agentes en ALE: Space Invaders

Brandon Werner Rivera Cabrera 23088

GUATEMALA, 14 de septiembre de 2026

Repositorio GitHub

<https://github.com/Brariv/Laboratorios-Deep-Learning/tree/main/lab5>

El Arcade Learning Environment

Qué es ALE y qué problema resuelve: El Arcade Learning Environment es una plataforma que expone decenas de juegos de Atari 2600 como problemas de aprendizaje por refuerzo con una interfaz común: recibe una acción del joystick y devuelve la pantalla, la recompensa y si el juego terminó. Resuelve el problema de evaluar agentes generales, porque un mismo algoritmo se prueba en muchos juegos sin conocimiento previo de cada uno usando la puntuación como recompensa. Está construido sobre Stella, un emulador de código abierto de la Atari 2600, y ejecuta las ROMs originales, por lo que la dinámica, los gráficos y la puntuación son las del juego de 1980.

Variantes de un mismo juego: La versión v5 fija los valores recomendados por Machado et al.: `frameskip` igual a 4, `repeat_action_probability` igual a 0.25 y `full_action_space` en False. `frameskip` es el número de frames consecutivos en que se repite cada acción.

`repeat_action_probability` es la probabilidad de que la consola ignore la acción nueva y repita la anterior, las llamadas sticky actions, que vuelven el entorno estocástico para que el agente no memorice secuencias. `full_action_space` decide si se exponen las 18 acciones del joystick o solo las que tienen efecto en el juego. La variante -ram cambia la observación de la imagen a los 128 bytes de memoria de la consola. En ale-py 0.12 ese sufijo ya no se registra como identificador separado y se obtiene con `obs_type` igual a ram, igual que `obs_type` igual a grayscale para la imagen en escala de grises.

Frame skipping: Atari corre a 60 frames por segundo y entre un frame y el siguiente casi nada cambia, así que decidir en cada frame es costoso y poco útil. Con `frameskip` igual a 4 el agente decide una vez cada 4 frames y la acción se repite en los intermedios. En el notebook, con `frameskip` igual a 1 el entorno simuló 6533 frames por segundo y con `frameskip` igual a 4 simuló 8657 frames por segundo tomando 2164 decisiones por segundo, porque los frames saltados no se renderizan ni se envían al agente. Para el aprendizaje, los episodios tienen cuatro veces menos pasos, el efecto de una acción es visible entre observaciones consecutivas y la recompensa debe propagarse por menos decisiones. El costo es menos capacidad de reacción dentro de esos 4 frames.

Space Invaders: El jugador controla un cañón que se mueve horizontalmente en la parte baja de la pantalla y dispara a una formación de 36 invasores en 6 filas que avanza de lado a lado y desciende, con 3 escudos que se desgastan con los impactos. Se gana al eliminar toda la formación y se cuenta con 3 vidas. El juego termina al perderlas o cuando la formación llega al suelo, y en ese caso se pierden todas las vidas restantes de una vez. Los invasores de la fila inferior valen 5 puntos y el valor sube 5 por fila hasta 30 en la superior, y la nave nodriza que cruza la parte alta vale hasta 200. ALE lee la puntuación de la memoria del juego y la recompensa de cada paso es la diferencia de puntuación acumulada en los 4 frames de la

acción, por lo que casi siempre es 0 y vale 5, 10 o 30 al destruir un invasor. No hay penalización por perder una vida ni recompensa por sobrevivir.

Espacios de observación y acción

Observación por defecto y comparación con CartPole-v1: ALE/SpaceInvaders-v5 devuelve un Box de enteros de 0 a 255 con forma 210x160x3, la pantalla RGB completa con 100800 valores por paso. CartPole-v1 devuelve un Box de 4 valores de punto flotante que ya son las variables de estado: posición y velocidad del carro, ángulo y velocidad angular del poste. Con imágenes el estado no viene dado y hay que extraerlo de los píxeles, lo que exige redes convolucionales en vez de un perceptrón pequeño, preprocesamiento para reducir la dimensión, mucha más memoria en el replay buffer y muchos más pasos de entrenamiento. Además un solo frame no contiene velocidades ni direcciones, por lo que la observación es parcial y se apilan frames para recuperarlas.

Observación en RAM: Con obs_type igual a ram la observación es un Box de 128 bytes con toda la memoria de trabajo de la consola, donde el juego guarda posiciones, contadores, vidas y puntuación. Conviene cuando el cómputo es limitado, porque basta un perceptrón multicapa y la simulación es más rápida, y cuando se quieren construir reglas o características interpretables. Sus desventajas son que el significado de cada byte cambia de juego a juego y no se transfiere, que la codificación puede ser difícil de aprender y que el agente deja de ver lo mismo que un humano.

Espacio de acción: Space Invaders usa el conjunto mínimo Discrete(6):

- NOOP: no hacer nada, el cañón se queda quieto.
- FIRE: dispara un proyectil, solo puede haber uno del jugador en pantalla a la vez.
- RIGHT y LEFT: mueven el cañón un paso a la derecha o a la izquierda.
- RIGHTFIRE y LEFTFIRE: mueven el cañón y disparan en el mismo paso.

Con full_action_space igual a True el espacio pasa a Discrete(18) con todas las combinaciones del joystick, incluidas UP, DOWN, las diagonales y sus versiones con disparo, que en este juego no tienen efecto y solo agrandan el espacio a explorar.

AtariPreprocessing y FrameStackObservation: AtariPreprocessing implementa el preprocesamiento estándar de DeepMind y Machado et al.: aplica un número aleatorio de no-ops al inicio del episodio, repite cada acción 4 frames haciendo max-pooling de los dos últimos para no perder los objetos que parpadean en frames alternos, convierte a escala de grises, redimensiona a 84x84 y opcionalmente marca la pérdida de vida como terminal y escala los píxeles a 0 y 1. El recorte de recompensa a -1, 0 y 1 no lo hace este wrapper, se aplica con TransformReward o dentro del algoritmo. Exige crear el entorno con frameskip igual a 1 para no saltar frames dos veces. FrameStackObservation apila las últimas 4 observaciones en un tensor de 4x84x84 para que el agente perciba movimiento y dirección, y se usa después de AtariPreprocessing porque con frames ya reducidos el apilado es barato en memoria. El notebook muestra las tres etapas: 210x160x3, 84x84 y 4x84x84.

Módulo de funciones

El módulo `ale_utils.py` implementa las funciones pedidas y el notebook las usa para generar los videos. `crear_entorno` recibe el nombre del entorno y parámetros opcionales que pasa a `gym.make`, y si se indica `video_folder` fija `render_mode` en `rgb_array` y envuelve el entorno con `gymnasium.wrappers.RecordVideo` con el `episode_trigger` y el `name_prefix` indicados, por lo que funciona con Space Invaders y con cualquier otro entorno de Gymnasium.

`agente_aleatorio` devuelve `env.action_space.sample()` y es la línea base. `agente_regla_simple` localiza el cañón y los invasores por su color en la imagen RGB, calcula el centro horizontal de la fila más baja de invasores y se mueve hacia él disparando, y cuando está alineado solo dispara. `ejecutar_episodio` reinicia el entorno, pide una acción a la función de agente en cada paso hasta que `terminated` o `truncated` sea verdadero o se alcance `max_steps`, y devuelve los pasos y la recompensa total. `generar_video_agente` crea el entorno con grabación, ejecuta `n_episodios`, llama a `env.close()` dentro de un bloque `finally` para que `RecordVideo` escriba los archivos y devuelve las rutas con las métricas. Videos generados en la carpeta videos:

Agente	Entorno	Video	Pasos	Recompensa
Aleatorio	ALE/SpaceInvaders-v5	aleatorio-episode-0.mp4	298	35
Aleatorio	ALE/SpaceInvaders-v5	aleatorio-episode-1.mp4	493	155
Aleatorio	ALE/SpaceInvaders-v5	aleatorio-episode-2.mp4	522	150
Regla simple	ALE/SpaceInvaders-v5	regla_simple-episode-0.mp4	1121	330
Regla simple	ALE/SpaceInvaders-v5	regla_simple-episode-1.mp4	1121	290
Regla simple	ALE/SpaceInvaders-v5	regla_simple-episode-2.mp4	1121	290
Aleatorio	CartPole-v1	cartpole_aleatorio-episode-0.mp4	18	18

Para comparar las dos políticas sin grabar video se corrieron 10 episodios adicionales de cada una con semillas fijas. El agente aleatorio obtuvo una recompensa promedio de 115.0 con desviación estándar de 52.5 y sobrevivió 445.8 pasos en promedio. La regla simple obtuvo 233.0 con desviación estándar de 98.8 y sobrevivió 1121 pasos en todos los episodios: nunca fue alcanzada por una bomba y sus episodios terminaron siempre en el mismo paso porque la formación llegó al suelo. Su puntaje queda limitado porque solo puede haber un proyectil del jugador en pantalla y muchos disparos se pierden contra los escudos, así que duplica al agente aleatorio pero no elimina la formación, y sirve como línea base contra la cual medir al agente entrenado del proyecto.

